React Native Client

Contents

React Native Client **1**

Scope . . . 1
Official requirements . . . 1
Project policy — the parts that bind the backend now . . . 1
Deferred — to define when client work begins . . . 2
References . . . 2
Related . . . 2

React Native Client

> **Status: deferred.** The mobile app is not in the current work scope (`brain/decisions/ADR-0002-mobile-react-nati`
> This standard records only the rules that constrain backend design today. Expand it when client work starts.

Scope

The React Native application that consumes the API. It lives at `MusicRoomApp/` in this same repository, owned by the UI developer (`brain/decisions/ADR-0020-monorepo-layout.md`). One clone is the whole submission, which subject VII requires.

The client generates its API types from `docs/api/openapi.json` with whatever tool its owner prefers; the backend publishes the document, not a client library.

Official requirements

- Subject IV.2: the mobile app must let the user perform every operation the project requires.
- Subject V.5: the app is a remote control for the backend, and **the backend address must be configurable in the app** for testing.
- Subject V.5: social network authentication (Facebook or Google) must work on mobile.
- Subject V.6: every action in the app creates a backend log with platform, device and app version.

Project policy — the parts that bind the backend now

1. **Configurable base URL.** Shipped as a setting, not a constant. The backend therefore must not generate absolute URLs from a fixed host, and its CORS allow-list is configuration-driven.
2. **Client context headers.** Every mutating request sends `X-Client-Platform`, `X-Client-Device`, `X-Client-App-Version`. These are the audit-log source (`../../60-quality/logging-observability/STANDARD.md`).
3. **No secrets in the bundle.** A React Native bundle can be unpacked. No YouTube API key, no OAuth client secret, no signing key ever ships in the app (`../../40-security/secrets-management/STANDARD.md`).
4. **Token storage.** Access and refresh tokens go in the platform secure store (Keychain / Keystore), never in `AsyncStorage`. This is why refresh tokens are rotating and revocable server-side rather than long-lived.

5. **Realtime client.** `@microsoft/signalr` is the supported client. The backend must keep the documented WebSocket handshake with the `access_token` query parameter available.
6. **Offline tolerance.** Treat every realtime event as a reconcile hint. Aggregate versions in payloads exist so the client can detect a gap and resynchronise over REST.

Deferred — to define when client work begins

`UNKNOWN - requires decision`: Expo vs bare CLI, TypeScript configuration, navigation library, state and data-fetching layer, component and styling conventions, accessibility targets (WCAG 2.2 AA as the intended baseline), client test strategy, release and OTA update process.

References

- React Native — https://reactnative.dev/docs/getting-started
- SignalR JavaScript client — https://learn.microsoft.com/aspnet/core/signalr/javascript-client
- WCAG 2.2 — https://www.w3.org/TR/WCAG22/

Related

`../../20-api/rest-api-design/STANDARD.md` · `../../50-integration/social-oauth/STANDARD.md` · `../../10-backend/signalr-realtime/STANDARD.md`